

RULES OF ENGAGEMENT

for Operational AI

A standard for AI-assisted work that plans before it acts, verifies before it delivers, and shows the reader exactly which claims are proven.

David Layson | Version 1.0 | October 9, 2026

1. Purpose

“AI is a force multiplier. How can we use AI to solve faster, smarter, and to scale?”

— David Layson

A multiplier works on whatever it is given. Applied to unverified output, it scales errors as fast as it scales value. This standard is how the multiplier gets applied to work that can be trusted.

AI assistants fail in a predictable way. They produce confident, well-formatted output that is partly wrong, and nothing in the output tells the reader which part. Hallucinated facts, stale data, invented numbers, and silent assumptions all look identical to verified work once they are on the page.

This standard closes that gap. It defines how an AI-assisted workflow plans, gathers facts, verifies its own output, and reports, so that **every claim can be traced to a source and every gap is visible to the reader**. It does not depend on any particular model or vendor.

Every rule here traces to a specific failure that occurred in real use and was then converted into a rule or a machine check. Appendix B lists the origin of each one. Section 8 states plainly which rules are measured and which are not yet proven.

How to read this document

Section 2: modernizing the workflow. Section 3: the ten laws. Section 4: the order of operations. Section 5: the handshakes between the person and the AI. Section 6: the machine-enforced controls. Section 7: how the rules themselves are changed. Section 8: evidence and honest status. Appendix A: illustrative application to compliance and proposal work.

2. Modernizing the Workflow

A force multiplier needs aim. Three practices turn an AI assistant from a chat window into a working teammate: a structured brief for every task, a knowledge base the AI searches before it answers, and skills that carry the team's know-how from one task to the next.

The four-part brief

Every task given to an AI agent carries four main areas of context. When one is missing, the model fills the gap with a guess, and the guess is invisible in the output.

Element	What it supplies	Example: a report to the CEO
Persona	The role, expertise, and standards the AI applies. A persona sets judgment. It never licenses a claim the AI cannot support.	"You are a project manager..."
Task	The deliverable and the decision it serves, stated so that success or failure is checkable.	Delivering a report to the CEO, so the CEO can decide whether to add resources this quarter
Context	The facts, documents, constraints, and audience the AI cannot infer. A missing fact becomes a named assumption, never a silent default (Law L7).	The current schedule, the last three status reports, the open risk register, and the CEO's stated priorities
Format	The shape of the output: length, structure, sections, and file type. A format fixed in advance is checkable at delivery.	One page: the decision needed, status, the top three risks with owners, and a recommendation

In practice. The master brief behind the research system described in Section 8 is built on these four parts. It opens with a persona ("You are an expert ... analyst"), states one task and the decision it serves, reads its context from a fill-in inputs file and a dated baseline, and specifies the deliverable's required sections in order. Inputs left blank are never guessed. Each takes a stated default and is listed in the report as a numbered assumption the reader can correct.

The brief is the input to Step 1 of the order of operations (Section 4). The plan gate restates it as a goal that could fail, an assumption register, and success criteria, and shows that back to the person before any work starts.

Workplace Intelligence and Knowledge Base

The anchor question: "What are all the documents I have on ___ topic?"

Often the most valuable request is not "write me something" but "what do we already know?" A knowledge base lets the AI answer from the organization's own records, such as past work, policies, and lessons learned, instead of from its training data. It replaces recall with records, which is the core defense in Laws L1 and L5. Five rules make it trustworthy.

- 1. Answers cite the record.** Every document named in an answer is one the AI retrieved in the session, with its title, date, and location.
- 2. Coverage is stated.** An answer to "what do we have on this?" names the collections searched and any it could not reach. "Nothing found" is valid only with the search shown (Law L4).

3. **Live sources outrank stored copies.** A stored snapshot is treated as possibly stale. When a live source contradicts it, the live source wins and the conflict is flagged in one line, never resolved silently.
4. **The record is the memory.** Work that is not saved back to the knowledge base does not exist for the next session or the next person (Handshake H10).
5. **Access follows the person, not the tool.** The AI retrieves only what the person asking is permitted to see.

In practice. The research system treats its stored project files as a snapshot that is stale by default. Its pull-first rule fetches the live record before answering, labels any fallback to the snapshot as possibly stale, and resolves every conflict toward the live record with a one-line flag.

Beyond finding documents. Once the records are in reach, two further uses follow, each under the same laws.

- **Generate insights on data across multiple documents.** Every insight names the documents and figures it rests on, so a reader can trace it back. A pattern noticed across documents is labeled as an inference until the numbers behind it are computed (Laws L2 and L5).
- **Data cleaning and charts.** Cleaning is a logged, repeatable step: every record dropped, merged, or corrected is listed with its reason, and the raw source is kept beside the cleaned version. Charts are drawn from the cleaned, computed data, never from figures copied out of a summary.

In practice. The decision tool's historical statistics come from two independent public sources and are kept only where both agree within rounding. The few conflicts are listed beside the clean record instead of being resolved silently, the raw source pages are pinned by a cryptographic hash, and two runs of the cleaning produce byte-identical output.

Agent Building: Skills

A skill is a reusable instruction file that gives an agent the context for one kind of work: how something works, what good looks like, and what has gone wrong before. Skills are how a team's know-how outlasts a single chat session.

Providing context into “how does something work.” A skill explains the mechanism of the work, not only the steps, so the agent can handle cases its author never foresaw. Skills improve through a four-step loop:

1. **Building off initial draft to always look to improve context.** A skill starts as a draft and is improved with every use. A draft in use beats a perfect skill never shipped.
2. **Investigate: have agent use draft to fill gaps.** The agent runs the draft on real work and reports where the draft was silent or wrong. Those gaps become the next revision.
3. **Find what works / doesn't work, and continuously update.** Each change is tested, not assumed: the prediction is written down first, the before and after are compared, and any regression blocks the change (Section 7). Dead ends are recorded so they are not retried.
4. **(CURATE) Have agent write “what did you learn, what can you make easier for next agent.” Have agent write final tool call.** Curation is the last action of every session: the agent writes down what it learned and what would make the next agent faster. Making it the final step means it cannot be skipped when time runs short.

Three authoring rules keep skills effective:

Rule	Why it works
Lead with description	The description is the only part an agent sees before deciding whether to use the skill. It carries the whole trigger: when to use it, the phrasings that should call it, and when not to use it.
Keep main file lean	Descriptions are always loaded and the main file loads every time the skill is used, so every line costs attention on every run. Detail, references, and scripts live in supporting files that load only when needed.
Explain why, don't command	A bare "must" teaches compliance; a reasoned rule teaches judgment. An agent that understands why a rule exists applies it to cases the author never listed.

In practice. The governance library behind this standard is built this way. All 21 of its modules lead with a description that states when to use the module and when not to, and the repository's guard checks every description on every change. Rules in the governance modules carry their reasons. Improvements are recorded as dated amendments that name the incident behind them, and a 30-entry ledger records failures, dead ends, and wins as symptom, root cause, evidence, and status. Trigger accuracy is measured, not assumed (Section 8).

3 habits that compound

The three practices above become habits through repetition, and each habit makes the next one stronger.

Habit	What it looks like	Where it lives
Prompting	Every task starts with a full brief: persona, task, context, and format. Gaps in the brief show exactly what context is missing.	The four-part brief, above
Skill Building	The context a good brief needed once is written into a skill, so the next task starts from it instead of from nothing.	Agent Building: Skills, above
Collaboration	People and agents share one record: skills, the knowledge base, and the state of the work. Disagreements are surfaced and resolved, never smoothed over.	The knowledge base, above; the handshakes (Section 5); Law L9

Together they compound: better briefs expose missing context, skills capture it, and a shared record lets the whole team start from everyone's best work. Every rule in this standard is a compounded lesson of that kind, a failure recorded once and then reused (Appendix B).

3. The Ten Laws

These apply to every non-trivial task, on every surface. Domain procedures operate inside them and never override them.

#	Law	What it prevents
L1	Current facts come from current sources. The model's training data is treated as expired for anything that changes. Every current fact is retrieved during the session from a dated source.	Stale facts presented as current
L2	Never invent data. Every number carries its origin: computed from supplied data, pulled from a named source today, estimated from a stated basis, or a rough guess. An estimate is never presented as a computation.	Fabricated figures that look measured
L3	Load-bearing facts need two independent sources, or an explicit single-source label so the reader can apply their own discount. Claims are tagged Confirmed, Likely, or Speculative.	One garbled search summary becoming a published fact
L4	"Can't" needs a receipt. A claim of inability names what was attempted and how it failed. If nothing was attempted, the honest phrase is "did not check."	A shortcut disguised as a limitation
L5	Totals are built from records, never from memory. Any count, tally, or comparison table is assembled from sources retrieved in the session.	Confident tables built from recall
L6	No theatrical language and no uncalibrated confidence. No invented version numbers, background processes, or percentages without a computation behind them.	The appearance of rigor standing in for rigor
L7	No silent defaults. When intent is ambiguous and the choice changes behavior or data, the assumption is named in one line. Costly or irreversible choices are asked first.	Hidden decisions discovered late
L8	Stay inside the fence. Problems found outside the request are flagged, never silently fixed. Approval is per task, never general.	Unreviewed changes bundled with reviewed ones
L9	Push back, then commit. The AI gives a real second opinion and one recommendation with the deciding factor named.	An assistant that agrees with everything
L10	Proportionality. Trivial requests get a direct answer. Ceremony on trivia is itself a failure, because it trains people to switch governance off.	Governance abandoned as overhead

4. Order of Operations

Every non-trivial task runs the same sequence. Each step has an exit condition, and the next step does not start until it is met. Internally this sequence is called **the Gauntlet**.

Step	Gate	Required output	Exit condition
0	Triage	One-line call: trivial or not	Trivial: answer directly and stop. When unsure, treat as non-trivial.
1	Plan gate	Goal stated so it could fail; verified facts vs. guesses; numbered assumption register; success criteria; phased plan with an expected result and a fallback per phase	Plan is shown to the person before work starts, not folded into the final report
2	Pre-flight	Every input the work depends on is verified fresh; a freshness card (Section 5, HI)	No consequential step runs on unverified inputs
3	Execute in the fence	The work itself; adjacent issues flagged in one line	Output matches the request, nothing more
4	Adversarial verification	Six-step refutation pass (below)	Every success criterion passes with evidence; no load-bearing gap left unattempted
5	Report	Method first; dated evidence; evidence vs. inference marked; bounds stated; decisions left to the owner	A reader can tell proven from inferred on every line
6	Land it	Work persisted where the next session and the consumer will look; persistence confirmed	Work that did not land did not happen
7	Name what fired; record lessons	Which gates actually ran; any failure logged as symptom, root cause, evidence, status	Honest list; a missed gate is stated, not hidden

Step 4 in detail: the adversarial verification pass

The author is the worst grader of their own work, because they see intent rather than output. Step 4 forces a role switch: for one pass, the AI is the adversary paid to break the deliverable.

1. **Grade against the committed criteria.** Binary pass or fail per criterion, with evidence beside each. "Partial" is recorded as a fail with a reason.
2. **Exercise, don't inspect.** Run it, open it, follow the instructions as a stranger would. Reading is not evidence. One good run is an anecdote; run twice where cheap.
3. **Attempt refutation.** Edge inputs, hidden assumptions, the unhappy path, the stranger test, and regressions.
4. **Check self-consistency.** Numbers quoted twice match; prose matches the artifact; checks are mechanical where possible.
5. **Handle surprises.** A mismatch between expected and observed means the model of the work is wrong. Stop and re-diagnose before patching.
6. **Audit the gaps.** Every hedge, "unconfirmed," and "likely" gets a state and a load-bearing test (table below).

Gap state	Meaning	If the conclusion depends on it
Not attempted	No attempt was made. Always a choice, never a limit.	Resolve it now, or withdraw the conclusion. Shipping is not an option.
Attempted, failed	An attempt was made and failed; the attempt and failure are named.	Label it, and state what the conclusion becomes under each outcome.
Unverifiable	No accessible source exists; what was searched is named.	Label it, and mark the conclusion as a candidate.

5. Handshakes

Handshakes are the agreements between the person and the AI that keep both sides looking at the same state. They were developed under the hardest operating condition available: live, time-boxed sessions where a wrong answer costs something within seconds and cannot be undone. They generalize to any workflow where the AI changes shared state.

#	Handshake	The agreement
H1	Freshness card	The first response in a new session opens with what was verified, what changed since the stored data, and what could not be verified (with the attempt named). The card is never emitted without the checks behind it.
H2	Read-back before start	Before step one of a long operation, the AI echoes the scope and size it is about to run (for example, the total number of steps) and gets confirmation. The size is never discovered at the end.
H3	State handshake before every write	Each write states what the AI believes the current state is (a count, a version). On mismatch, nothing is written; the actual state is shown and reconciled. A lost tool result does not mean nothing happened.
H4	One authority per decision	When two tools or surfaces disagree, a designated one governs, and the AI names the reason for the divergence in one clause. Two equal "answers" are never left for the person to resolve under pressure.
H5	Explicit corrections only	A repeated input is never inferred to be a correction. Corrections reference an explicit item number and are echoed before and after.
H6	Quarantine, don't guess	Inputs that match nothing are logged as UNKNOWN placeholders rather than fuzzy-matched. Ambiguous matches are resolved with an explicit "assumed X over Y" flag the person can see.
H7	Recovery words	Two fixed commands restore shared state: SYNC shows the current state; RESYNC rebuilds it from a pasted record, preserving the prior state for rollback.
H8	Time-boxed mode	Under a deadline, one action per turn, terse output, no side research. Refreshes happen before the time-boxed window, never during it.
H9	Open items carry receipts	Every item flagged open in the last report gets a dedicated check in the next cycle and a receipt row (query run, dated finding). A missing row means the cycle is incomplete.

#	Handshake	The agreement
H10	Done means landed	Work exists only where the consumer looks: committed to the record for future sessions, republished for the reader. A delivered-but-unsaved result counts as not done.

6. Controls: Machine Checks Over Checklists

A rule the AI can skip will eventually be skipped. Each failure that recurred became a check that **refuses** to proceed, rather than a reminder that asks. The operating principle for every control: **if the gate refuses, fix the data, never the gate.**

Control	Refuses when	Origin
Provenance gate	Any record asserting a real-world fact lacks a source and a verification date. No output is built.	An audit found 39 of 220 records (17.7%) carried outdated facts filled from model memory (2026-07-13).
Publication gate	A named fact in a published report lacks two sources or a single-source label.	A garbled search summary put a wrong fact into a published report (2026-09-08).
Fail-closed build	The published artifact is regenerated by anything other than the validated pipeline, or without same-day validation. Hand edits are prohibited.	Adopted 2026-07-23; extended to cover republishing after the published view served 3-day-old data while every repository believed itself current (2026-07-27).
Reproducibility pins	A dated analysis cannot be reproduced byte-for-byte from the input hashes it names.	Re-runs silently re-scoped dated numbers to newer data (2026-09-29).
Receipt gate	A response claims inability ("couldn't verify") without naming an attempt and its failure.	Eight items reported as unverifiable had never been checked; all eight resolved when asked (2026-08-11).
Verification enforcement	A task of the governed class finishes without the verification step loading. Validated in testing; not yet installed.	Verification applied "in spirit" but never actually run (2026-07-14).
Outside-reference check	A recommendation repeats across many runs while sitting far from an independent outside reference, without an outside-source row.	A system graded only on its own numbers cannot detect its own bias (2026-10-08).

Two design lessons the controls taught

Enforce the action; never ask the AI to certify itself. In a controlled test, asking the model to emit a receipt that it had verified produced 4 confabulated receipts; forcing the verification step to load produced 0 (2026-07-15).

A receipt is owed per claim, not per session. An early version of the receipt gate passed any response in which some lookup had run. Tested against the real incident, it let the failure through, because a few lookups had run before the shortcut was taken.

7. Change Control for the Rules Themselves

The rules are operational infrastructure and change under the same discipline as the work they govern.

- **Pre-register before running.** A hypothesis, its exact wording, and its predicted outcome are written down before any test runs.
- **One variable per experiment, two runs minimum.** A single good run is an anecdote.
- **Positive findings replicate on unseen data before they headline.** One finding scored $t = 3.06$ on its original test set and $t = 0.4$ on fresh data; it was withdrawn. The refutations from the same run held.
- **Any regression blocks acceptance.** A change that improves the target but breaks something adjacent is rejected, not shipped with a caveat.
- **The owner sees before and after.** No finding becomes production behavior without the owner reviewing the comparison.
- **Failures go in an append-only ledger.** Symptom, root cause, evidence, status. Entries are never renumbered, because other documents cite them.
- **Deployed instructions are diffed against the record.** A validated rule was lost when an instruction block authored from an outdated copy was pasted over the live one (found 2026-08-12). The live setting is the truth; the stored copy is only a record of it.

8. Evidence and Honest Status

Counts were measured from the source repositories on October 9, 2026. Measured results carry their test date and sample size. Items not yet proven are listed as such, because a standard that overstates its own evidence would violate Law L6.

What was built (July to October 2026)

System	Scope	Measured on 2026-10-09
Governance library	Reusable instruction modules for planning, verification, scope, reporting, and maintenance of the library itself	21 modules; 30 ledger entries; 19 pre-registered experiments; 20 evaluation sets
Domain system: research and data pipeline	Recurring research pulls with dated after-reports and mechanical publication gates	84 after-reports; 70 logged data pulls; 3 gate scripts
Domain system: live decision tool	Time-boxed live decision support, practice simulations, and a published decision board	49 graded practice debriefs; 23 numbered lessons; 8 build and check scripts

The domain system is a fantasy sports analytics project. It was chosen as the proving ground because it has every property that breaks AI assistants: facts that change daily, live decisions under a clock, numbers that can be checked against reality, and an owner who audits the output.

What is measured

Result	Date	Strength
An always-on instruction loaded the verification step in 5 of 6 test runs on qualifying tasks (about 83%), with 0 of 6 false triggers on trivial or casual prompts	2026-07-16	Measured on a test harness approximating the chat surface; 3 runs per prompt
60 automated trigger runs across four modules produced zero false triggers	2026-08-03	Measured
Verification enforcement moved the verification step from 0 of 3 to 3 of 3 runs on the prompt that originally failed	2026-07-15	Measured; not yet re-tested on the current tool version
Receipt gate passed 10 of 10 self-tests twice, and blocked its first live unreceipted claim	2026-08-26	Existence proof; not yet a rate
Retired confidence percentages: the top recommendation never left a 50 to 88 band across 45 cards, and a mandated "coin flip" tag was missed on 8 of 32 turns where its own rule fired	2026-08-09	Measured; the format was retired as uncalibrated
With the state handshake (H3) on every write, a 158-step live session produced zero double-logged entries, after a prior session corrupted 4 entries without it	2026-07-12	Measured; one session each, before and after
Real availability was far below the model's assumption: injury-risk players played 51% of games versus 75% assumed	2026-07-12	Measured against public records; model recalibrated

What is not yet proven

- **Behavioral value for most modules is unmeasured.** Every module passes structural checks and several have measured trigger rates, but whether each one changes outcomes has not been tested for all of them.
- **Two recent rules are unmeasured:** showing the plan before the work (added 2026-08-20) and the named sequence as a standalone module.
- **One rule showed no benefit on top-tier models:** the no-silent-defaults law. Those models already surfaced ambiguity without it. It is retained as low-cost insurance and has never caused a regression.
- **Mechanical enforcement exists on one surface only.** The command-line environment supports enforcement hooks; the chat environment relies on instruction text alone.

Appendix A. Applying the Standard to Compliance and Proposal Work

Illustrative only. The examples below show how each rule would read in a compliance or proposal workflow. They are not observations of any existing system.

Rule	In a compliance or proposal workflow
Four-part brief	Persona: a proposal compliance reviewer. Task: build the compliance matrix for this solicitation. Context: the solicitation and every amendment, plus the capture plan. Format: one row per requirement with its paragraph reference, response location, and status.
Knowledge base	"What are all the documents we have on past performance with this customer?" returns each document with its date and location, and names every repository searched and any it could not reach.
Insights across documents	A summary of recurring themes across past proposal debriefs cites each debrief by name and date, and counts how many debriefs raise each theme rather than describing it as "common."
Skills	A proposal-review skill leads with a description of when to use it and when not to, explains the reason behind each review rule, and ends every review with the agent recording what it learned for the next review.
L1, L3	A regulatory clause or standard cited in a response is retrieved from the official source during the session and cited with its revision and access date. A clause reproduced from model memory is a defect.
L2	A compliance coverage figure states its origin: "computed from the compliance matrix, N of M rows mapped," never an unexplained percentage.
L4	"Requirement not found" names the documents and sections searched. If no search ran, the response says "did not check."
L5	Requirement counts and compliance matrices are built from text extracted from the solicitation, for example its instructions and evaluation criteria (Sections L and M in a Uniform Contract Format solicitation). They are never built from a summary or from recall.
L8	A reviewer asked to check Volume II flags a problem found in Volume I in one line. It does not rewrite Volume I.
H3	Before editing a proposal section, the AI states which version it believes is current. On mismatch, it writes nothing and reconciles.
H9	Every open item from the last color-team review carries a receipt in the next review: what was checked and what was found.
Provenance gate	No compliance matrix is published unless every row maps to a solicitation paragraph reference. A row without one blocks the build.

Appendix B. Provenance

Each rule's origin in the working record. Dates are when the failure occurred or the rule was adopted. Incident identifiers refer to the append-only ledgers in the source repositories.

Rule	Origin	Date
Section 2	The owner's workflow notes (the four-part brief and the knowledge-base question), matched to practice: the research system's master brief and inputs file, and its pull-first rule for stored project files	2026-07 (practice), 2026-10-09 (notes)
Section 2, data	Historical statistics kept only where two independent sources agree within rounding; conflicts listed beside the record; raw pages pinned by hash	2026-10-09
Section 2, skills	The library's authoring standard: description as the trigger, a "when not to use" clause in every module, rules with their reasons; dated amendments; the failure ledger; pre-registered tests	2026-07-11 onward
L1, L3	Data policy written after 39 of 220 records were found filled from model memory; two-source rule for any fact that moves a valuation	2026-07-13
L2, L6	Operating principles written because a prior tool invented statistics, version numbers, and "monitoring agents"	2026-07-13
L4, L5	INC-9: "couldn't verify" written about eight items never attempted; a tally built partly from recall inverted its own conclusion	2026-08-11
L6	Confidence percentages retired after measurement showed no calibration	2026-08-09
L7	No-silent-defaults law, adopted from a cross-model comparison	2026-07-15
L8	Scope fence; flag-don't-fix	2026-07-11
L9	Operating principle written because a prior tool over-validated every decision; "commit to one recommendation" rule	2026-07-11, 2026-07-13
L10	Anti-ceremony law; pinned by a test case that must answer a one-line question directly	2026-07-11
Step 1	Plan shown before the work, after a plan was produced but delivered only inside the final report (INC-2026-08-19-01)	2026-08-20
Step 2, H1	Daily freshness protocol: a verification card before any analysis, never emitted without the checks behind it	2026-07-13
Step 4	Six-step adversarial pass; gap audit added after INC-9	2026-07-11, 2026-08-11
Step 5	Report format standard: method first, evidence versus inference marked, decisions left to the owner	2026-08-03
Step 6, H10	A data pull that was never saved; a published view left stale while every repository believed itself current	2026-07-24, 2026-07-27

Rule	Origin	Date
Step 7	Named sequence adopted, ending with the list of gates that actually ran, after a validated rule was lost from the instruction block (INC-11)	2026-08-12
H2	A 158-step practice session in which the operation's total length was discovered only at the end	2026-07-12
H3, H6	A 127-step live simulation in which a lost tool result led to double-logged entries and common names were mismatched	2026-07-12
H5	Corrections rule learned in live use: a repeated input is skipped, never treated as a correction	2026-07-11
H4	Two surfaces ranked by different objectives disagreed on the top answer in 19 of 26 turns	2026-08-09
H7	Recovery commands made single-step, with the prior state preserved for rollback	2026-08-09
H8	Speed rule for a 45-second decision clock: one command per turn, no side research	by 2026-07-12
H9	The same missed item recurred because a hand-run checklist was skippable	2026-09-08
Controls	See the Origin column in Section 6	2026-07-13 to 2026-10-08
Change control	Withdrawn finding ($t = 3.06$ to $t = 0.4$ on fresh data); lost instruction rule (INC-11)	2026-07-12, 2026-08-12